

LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous submission

Abstract

Sparse mixture-of-experts (MoE) models reduce arithmetic per token but retain a large parameter footprint. Expert offloading can fit such models on a memory-constrained accelerator, yet conventional offloading changes weight locations on the decode path. This behavior conflicts with graph replay, which relies on stable addresses and a repeatable execution topology; disabling replay exposes a host-bound decode path.

We present LatchMoE, an expert-offloading runtime that separates dynamic expert residency from captured tensor addresses. LatchMoE preallocates fixed device slots, stages routed experts outside the replay boundary, protects slots with an explicit transfer/compute lifecycle, and executes oversized prefill working sets in capacity-bounded waves. The resulting data plane accepts placement plans without embedding a particular prediction policy.

The implementation targets a hook-enabled vLLM-Ascend stack and includes host-side correctness tests and a manifest-oriented online benchmark harness. Historical capture-disabled diagnostics lack the raw, repeated evidence needed for a paper claim and are excluded from the submission matrix. We therefore state the mechanism and correctness claims independently and require repeated graph-mode online comparisons against matched graph-compatible baselines before making a performance claim.

1 Introduction

Mixture-of-experts (MoE) models increase parameter capacity while activating only a small subset of experts for each token [1, 2, 4, 6, 11]. Sparse activation reduces the arithmetic performed per token, but it does not remove the need to make every routed expert available. Consequently, expert weights can dominate the memory footprint of an MoE deployment. Distributed systems spread this capacity across accelerators [10], while heterogeneous inference systems coordinate model storage across device and host memory [13].

For a memory-constrained single accelerator, expert offloading applies the latter idea specifically to MoE layers: the runtime keeps a bounded set of experts in high-bandwidth memory (HBM) and fetches non-resident experts from host memory as routing demands them [3, 16]. This exchange makes otherwise infeasible model capacity accessible, but it also places cache lookup, weight movement, and replacement on the serving path.

Prior systems reduce this data-movement cost through expert prediction, prefetching, caching, mixed precision, and finer-grained offloading [5, 7, 9, 14, 15, 18]. These mechanisms improve which experts are moved and when they arrive, yet routing-driven cache misses still change the correspondence between logical experts and physical weight storage. A separate line of compiler and serving work addresses model dynamism and graph-compatible scheduling [8, 12, 17]. Accelerator graph replay amortizes repeated host dispatch, but the replayed region assumes a repeatable operator topology and stable graph-visible storage bindings. Expert replacement violates this assumption even when tensor shapes and kernel choices remain unchanged: the same logical expert may occupy a different allocation at the next decoding step. Eager execution can resolve the new placement before every kernel launch, whereas excluding the entire MoE path from capture reintroduces fine-grained host orchestration. Data-movement optimizations alone therefore do not provide a replay-stable interface for dynamic expert residency.

Our goal is to preserve graph replay while allowing the resident expert set to change within a fixed HBM budget. The key observation is that expert *identity* may change without changing the addresses consumed by captured kernels. A captured MoE computation does not need each logical expert to keep its original allocation; it needs fixed physical storage containing the right weights and a consistent logical-to-physical mapping at replay time. This observation turns dynamic residency into an inter-replay state transition: weight contents and mapping values may be updated at an explicit replay boundary, while the slot tensors and mapping buffers

visible to the graph keep their addresses. It also separates the data plane that enforces residency from the control policy that prioritizes expert placement.

Realizing this separation raises three challenges. First, the fused MoE operator hides the routing decision inside an opaque computation, leaving no graph-visible boundary at which data-dependent staging can legally execute; exposing such a boundary must not change routing or the tensors passed between captured regions. Second, asynchronous transfers and expert computation share the same bounded slot bank. A new load must not overwrite a slot used by in-flight compute, and a mapping must not expose a new owner before its transfer has completed. Third, a prefill or mixed batch can activate more experts than the slot bank can hold simultaneously. Full residency would violate the HBM budget, while dropping routed experts or approximating their outputs would change model semantics. The runtime must therefore execute an oversized working set exactly in capacity-bounded phases while retaining opportunities to overlap transfer and computation.

We present LatchMoE, a graph-compatible expert-offloading data plane for vLLM-Ascend. LatchMoE decomposes the opaque MoE forward path into a captured router, an eager staging seam, and a captured expert MLP. At the seam, it reads the active expert set, applies a placement plan, loads misses into a preallocated slot bank, and updates a persistent logical-to-physical mapping in place; the captured MLP continues to consume the same slot and metadata addresses. Each slot carries an owner and version, and transfer and compute events protect that ownership until all users of the version have completed. When an active set exceeds capacity, LatchMoE partitions routed token-expert pairs into exact waves, stages and computes one bounded wave at a time, and restores outputs to their original token positions. Multiple staging buffers permit the next wave’s transfers to overlap the current wave’s compute. An HBM- and KV-aware startup configuration selects resident layers and slot capacity, while a policy-neutral placement-plan interface allows external controllers to reuse the same data plane.

In summary, this paper makes four contributions:

- We characterize the address-binding conflict between dynamic expert residency and accelerator graph replay, and derive the requirements for a replay-stable offloading data plane (§2).
- We design LatchMoE’s three-stage decomposition with a captured router, eager staging seam, and captured MLP, together with fixed-slot expert virtualization. These mechanisms move data-dependent placement outside captured regions while preserving stable graph-visible addresses (§§3–4).
- We introduce a versioned, compute-protected slot life-cycle and exact capacity-bounded wave execution for working sets that exceed slot capacity (§§3–4).
- We implement LatchMoE as a vLLM-Ascend platform plugin and define a graph-only evaluation methodology that records graph execution, HBM use, expert transfers, and service metrics while rejecting eager fallback as a successful run (§5).

2 Background and Problem

2.1 MoE residency under limited memory

An MoE layer contains E experts but routes each token to only $k \ll E$ of them. Sparse activation reduces compute, as in Switch Transformers [4], but not the capacity needed to make every expert available. Offloading exploits temporal sparsity: inactive weights remain in host memory, and the runtime caches or prefetches experts into a bounded device-resident set. Systems such as MoE-Infinity use activation traces to improve these decisions [16]; FlexGen demonstrates the broader value of coordinating model storage across heterogeneous memory [13].

The runtime cost is determined not only by bytes transferred. Demand loads can stall a decode step, prefetches can contend with computation, and residency updates can add host work. For graph-based execution, residency also interacts with capture validity.

2.2 Why ordinary offloading breaks replay

Graph replay records a repeated sequence of operations and their tensor relationships. A naive expert cache materializes the selected expert in a new buffer or mutates the weight pointer used by the MoE kernel. Both approaches make the repeated decode iteration depend on dynamic host actions inside the captured region. Falling back to eager execution restores flexibility but removes the optimization we seek to preserve.

We require four properties:

1. **Address stability:** captured kernels reference a fixed set of device allocations.
2. **Ownership safety:** a slot has one logical expert and cannot be overwritten while compute uses its current generation.
3. **Capacity safety:** no route may silently exceed the configured slot budget.
4. **Observable replay:** graph capture, replay, and fallback events are recorded and cannot be replaced by a forced eager success.

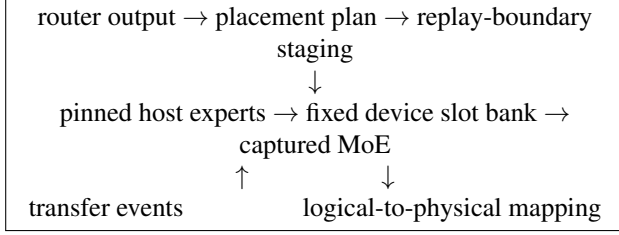


Figure 1: LatchMoE keeps tensor addresses stable and moves dynamic routing into metadata and transfers outside the captured region.

3 Design

3.1 Fixed-slot expert virtualization

At initialization, LatchMoE allocates a fixed number of equal-shaped expert slots. A slot has a stable tensor address, an owner (ℓ, e) consisting of layer and expert identifiers, and a monotonically increasing generation. The logical-to-physical map translates a routed expert to its current slot. Captured grouped matrix multiplications consume slot tensors and indices, not fresh expert allocations.

The slot count is derived from the offload target, physical HBM, a KV-cache reserve, and a minimum net-saving constraint. Explicit capacity checks reject an active working set that cannot be represented by the current execution mode. This prevents an apparently successful run from borrowing unaccounted memory.

3.2 Replay-boundary staging

The router produces active expert identities. A placement-plan adapter computes retain, evict, demand-load, and optional prefetch sets. The adapter is deliberately policy-neutral: LRU or predictive control can be supplied by a separate controller without changing the slot bank.

Staging occurs before the captured MoE region. Host weights are copied on a dedicated transfer stream, and the mapping is published only after the corresponding load completes. The graph then replays against stable slots. No transfer or allocation is hidden inside the captured kernel sequence.

3.3 Compute-protected lifecycle

Each slot progresses through an ownership protocol: free, loading, ready, computing, and eviction-pending. Loading establishes a new owner and generation. Compute records the generation it consumes and protects that slot until its completion event. Eviction waits for both transfer and compute dependencies. Shutdown drains pending work before releasing storage.

The central safety invariant is:

A mapping may expose expert (ℓ, e) at slot s only if s is ready, owns (ℓ, e) , and its published generation equals the generation checked by the consumer.

Violations fail closed rather than selecting an eager path.

3.4 Capacity-bounded wave prefill

Prefill can route many tokens and activate more experts than the slot bank can hold simultaneously. LatchMoE partitions routed work into waves whose expert set fits the bank. For each wave, it stages missing experts, computes the corresponding token subset, and restores output order before combining results. With multiple staging buffers, the next wave’s transfers can overlap the current wave’s compute.

Wave construction preserves token-to-expert assignments; it changes execution order, not routing semantics. Correctness tests compare the scattered and restored output against the unpartitioned result.

4 Implementation

LatchMoE is a Python package registered through vLLM’s platform-plugin entry point. The implementation contains the slot bank, pinned host store, transfer engine, prefill wave scheduler, automatic capacity configuration, profile events, and fused-MoE adapters. A hook-enabled vLLM-Ascend fork exposes the boundaries at which routing, staging, and graph state can be coordinated.

The integration leaves request scheduling and the OpenAI-compatible API unchanged. Disabling the plugin restores the runtime’s null hooks. Native weight-offload flags cannot be combined with LatchMoE because two independent owners of expert storage would violate the slot invariant.

5 Evaluation

We separate implementation evidence from the experiments required to support a submission claim.

5.1 Current evidence

The repository includes host-side tests for slot ownership, phase splitting, wave restoration, graph guards, and benchmark artifact collection. These tests validate contracts but do not measure online performance. Historical capture-disabled diagnostics are excluded because they lack the raw, repeated bundle required for a paper claim and violate the graph-only evaluation boundary. The current draft therefore makes no online performance claim.

5.2 Submission experiment matrix

The checked-in harness defines real ShareGPT prompt buckets, graph-only cases, 14- and 28-GiB budgets, and artifact-preserving failure outcomes. The final evaluation must answer four questions:

- Q1: Feasibility.** Does full residency fail under the same HBM/KV reservation for which LatchMoE serves requests successfully?
- Q2: Graph compatibility.** Do capture and replay complete, and what fraction of decode steps replay without fallback?
- Q3: Performance.** How do TTFT, TPOT, inter-token latency, throughput, and failure rate compare with matched graph-compatible native and legacy offload baselines?
- Q4: Attribution.** What do fixed slots, asynchronous loading, transfer ordering, wave prefill, and automatic capacity selection each contribute?

Each cell requires three independent service starts and must report median and dispersion. Results must carry parent/runtime revisions, device identity, model and workload digests, graph mode, HBM accounting, raw client metrics, server logs, and transfer profiles. Formal runs may use only an explicitly reserved and idle physical NPU5 or NPU6. A graph failure is retained as a failed artifact and is not rerun with forced eager execution.

6 Discussion and Limitations

Policy is intentionally separate. LatchMoE supplies the data plane and a generic plan interface. Predictive placement, route-transition models, and EPLB comparisons belong to a separate control-plane study. This boundary allows all policies to share identical slots, transfers, and graph behavior.

Current scope. The validated configuration is a single low-concurrency device and one unquantized MoE model. Multi-device expert parallelism changes the comparison because its transfers are device-to-device rather than host-to-device. Higher concurrency can also change active-set size and overlap opportunities.

Evidence gap. The absence of checked-in raw artifacts is the primary submission blocker. Until the repeated matrix is complete, the paper supports a mechanism and correctness argument, not a general performance claim.

7 Related Work

MoE architectures use sparse routing to increase parameter capacity without activating every parameter per token [4]. DeepSpeed-MoE combines model and parallelism techniques for large-scale MoE inference [10]. MoE-Infinity focuses on activation-aware caching and prefetch for expert offloading [16]. LatchMoE addresses a different constraint: keeping the offload data plane compatible with captured decode execution through stable slot addresses and an explicit lifecycle.

Heterogeneous inference systems such as FlexGen coordinate storage and execution across device, host, and disk memory [13]. vLLM uses paged KV-cache management and continuous batching to improve LLM serving [8]. LatchMoE is complementary: it virtualizes expert weights rather than KV blocks and preserves the repeated MoE execution graph while residency changes.

8 Conclusion

LatchMoE decouples dynamic expert identity from the stable addresses required by graph replay. Fixed slots, replay-boundary staging, protected ownership, and wave prefill form a graph-compatible offload data plane. The current artifact establishes the design and its host-side contracts; repeated, manifest-backed online evaluation remains necessary to establish the breadth and magnitude of its performance benefit.

References

- [1] Damai Dai, Chengqi Deng, Chenggang Zhao, Ruijian Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yongle Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [2] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. *arXiv preprint arXiv:2112.06905*, 2021.

- [3] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 2022.
- [5] Ranggi Hwang, Jianyu Wei, Shijie Cao, Chang Ho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pregated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.
- [6] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lelio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Theophile Gervet, Thibaut Lavril, Thomas Wang, Timothee Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [7] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [9] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. CommitMoE: Efficient fallback-free MoE inference with offloading under GPU memory constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [10] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. *arXiv preprint arXiv:2201.05596*, 2022.
- [11] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [12] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proceedings of Machine Learning and Systems*, 2021.
- [13] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [14] Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [15] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [16] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [17] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie W. Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *arXiv preprint arXiv:2501.01005*, 2025.
- [18] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. Taming latency-memory trade-off in MoE-based LLM serving via fine-grained expert offloading. *arXiv preprint arXiv:2502.05370*, 2025.